LA4VLA: Learning to Act without Seeing via Language-Action Pretraining Note

Navigation

- Paper card: LA4VLA: Learning to Act without Seeing via Language-Action Pretraining card

Reading Notes

视觉-语言-动作（VLA）模型通常通过联合映射视觉观察和语言指令到动作，在机器人演示数据上进行预训练。然而，**密集的视觉动作监督可能会掩盖相对稀疏的语言-动作信号**。因此，策略可能依赖于视觉捷径，而不是学习语言如何调控动作执行，从而使其对视觉变化非常敏感。为了解决这一局限性，我们提出了LA4VLA，一个语言-动作预训练框架，它使策略能够在没有视觉观察的情况下获取语言调控的动作先验。这些先验**捕捉了跨任务和场景共享的可重用操作技能，从而减少了对特定场景视觉线索的依赖**

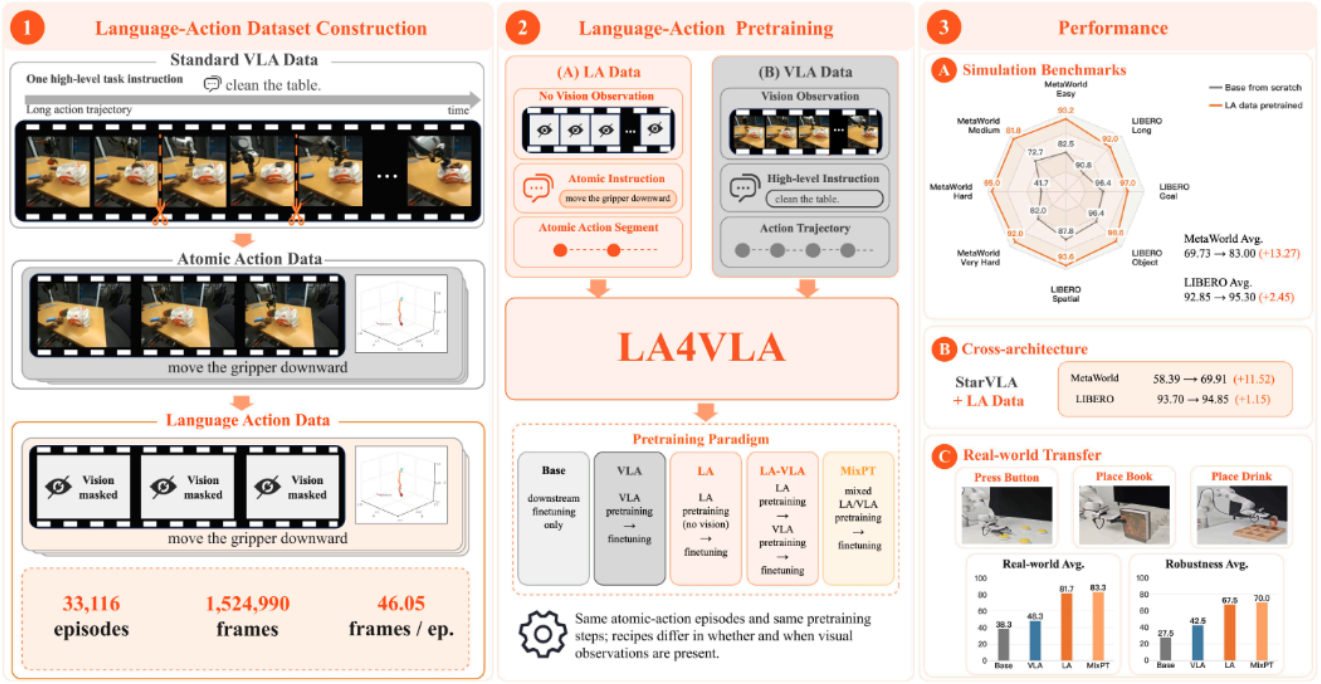
视觉和语言对动作学习的贡献方面存在固有的不对称性：

- 在数据层面，一条机器人轨迹可能包含数百甚至数千个图像-动作对，但通常只与一条高级任务指令配对
- 在模型输入层面，视觉观察通常被编码成数百个图像标记，其数量通常比语言指令的标记数量多一到两个数量级

做法：在预训练阶段去掉视觉观测

LA4VLA，这是一个构建显式“语言-动作”（LA）监督信号并将其融入 VLA 学习的预训练框架

结论：仅使用 LA 数据进行预训练所带来的下游任务性能提升，优于仅使用 VLA 数据进行预训练的效果；而结合 LA 和 VLA 数据则能带来进一步的性能增益。语言-动作监督为下游 VLA 策略提供了一种有效且可迁移的训练信号，并与现有的 VLA 学习流程形成了互补。



Probing Instruction Following in VLAs

普通 VLA 看起来会听语言，但这种“听语言”到底是真的语言控制，还是因为图像和语言在训练数据里刚好配对？

假设原始样本是：

```
Instruction: "move upward to approach the target"
Robot state: s_t
Paired image: 图像中对应的是“向上移动”的轨迹阶段
```

A. Standard paired input

```
输入 = 原始 instruction + 原始 robot state + 原始配对 image
```

B. Visual-removed input

```
输入 = 原始 instruction + 原始 robot state + mask 掉 image
```

C. Visual-unaligned input

```
输入 = 原始 instruction + 原始 robot state + 同场景但不配对的 image
```

D. Visual-conflict input

```
输入 = 原始 instruction + 原始 robot state + 相反方向动作对应的 image
```

Metric：

DAR：Directional Alignment Rate

DAR = 预测终点位移是否落在 instruction 指定方向的半空间里。

例子：

```
Instruction: move upward
预测位移: Δz > 0  → 记为对
预测位移: Δz < 0  → 记为错
```

DCS：Direction Consistency Score

DCS = 预测终点位移和真实 demonstration 位移之间的 cosine similarity。

例子：

```
真实动作: 向上 + 稍微向右
预测动作: 也向上 + 稍微向右
DCS 接近 1

真实动作: 向上
预测动作: 横向乱动
DCS 接近 0

真实动作: 向上
预测动作: 向下
DCS 接近 -1
```

SR：Separability Ratio

SR = 不同方向命令之间的平均距离 / 同一方向命令内部的平均距离。

直观理解：

```
move up 的预测都聚在一起
move down 的预测都聚在一起
而且 up 和 down 两组相距很远
→ SR 高
```

SS：Silhouette Score

SS = 聚类质量指标，看每个样本是否更靠近同类 direction cluster，而不是其他 direction cluster。

例子：

```
move up 的点大多靠近其他 move up 点
并且远离 move down / left / right 点
→ SS 高
```

SS和SR都是基于末端位移量来算的：对每个预测 action chunk，先把 horizon 内的平移增量加起来，就是这次预测的末端总位移向量。

结果：

条件	DAR	DCS	SR	SS	解释
Standard paired input	0.98	0.95	2.35	0.55	正常配图时，看起来非常听指令
Visual-removed input	0.63	0.16	1.03	0.04	去掉图像后，方向一致性严重下降
Visual-unaligned input	0.66	0.37	1.13	0.05	换成不配对图像后，语言控制仍然很弱
Visual-conflict input	0.35	0.03	1.03	-0.04	冲突图像会把动作带偏到相反方向（DAR<0.5）

可视化：

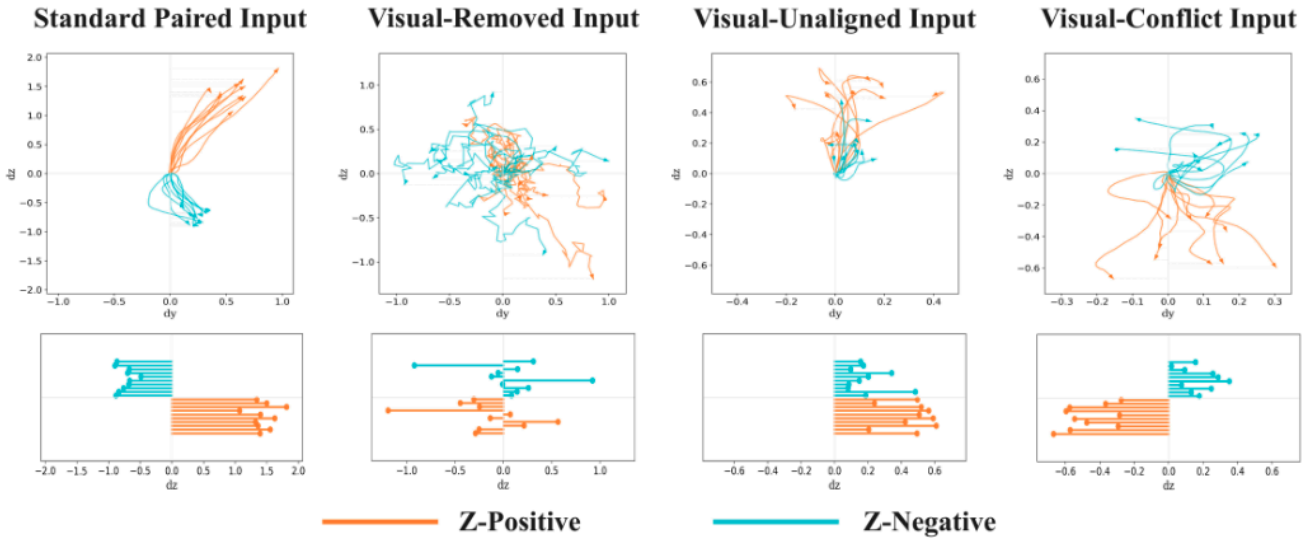
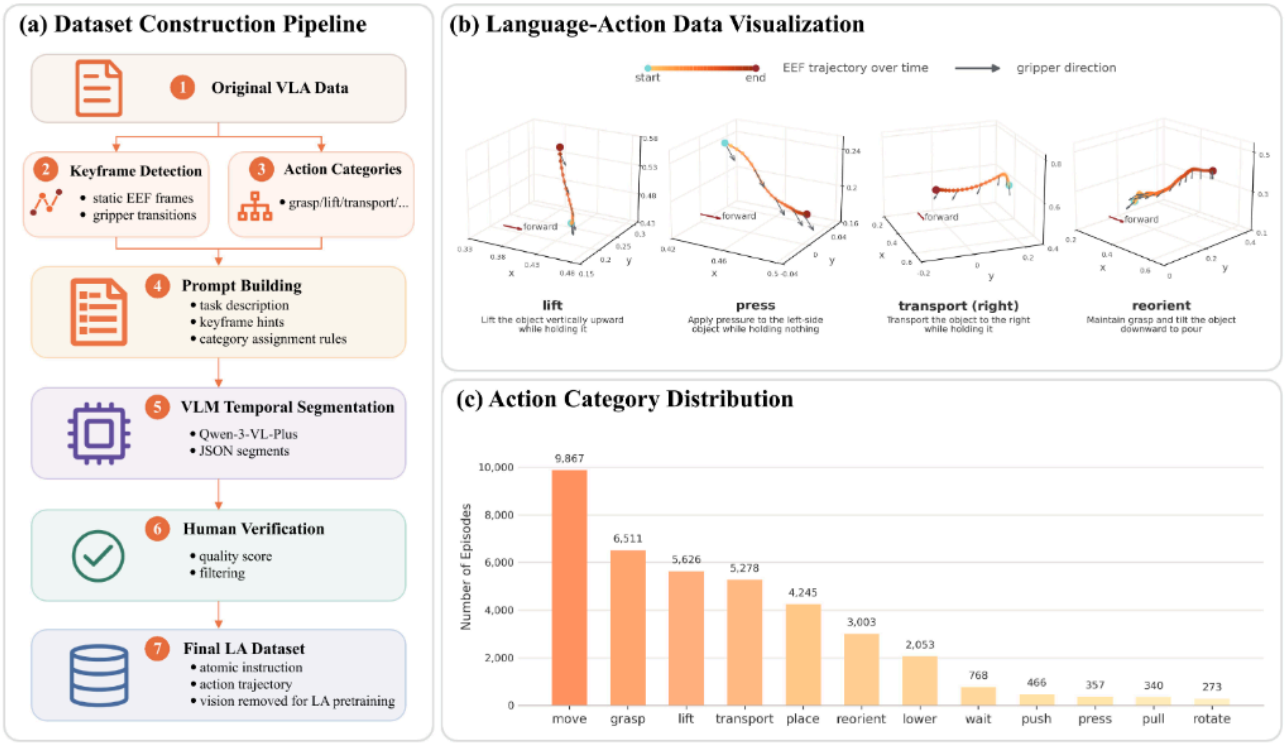


Figure 2 Qualitative visualization of instruction-conditioned direction following under different visual input configurations. The top row shows predicted trajectories projected onto the plane containing the tested motion axis, and the bottom row presents endpoint displacement projected onto that axis.

Language-Action Dataset Construction



这部分讲的是如何构建LA dataset: 把原来稀疏的高层语言监督，变成更密集的 atomic language-action supervision

Step 1：从原始 VLA 轨迹开始

原始 demonstration 里有：

```
visual observations
robot states
language instruction
action trajectory
```

例如原始任务级指令可能是：

```
put the cup into the bowl
```

这句话太高层，不能直接作为每个局部动作片段的语言监督。论文要做的是把一条长轨迹切成短的 atomic segments，并给每段配一个低层动作描述。

Step 2：提取 keyframe hints

他们先从 robot state 里提取时间提示：

```
static keyframes
gripper-open events
gripper-close events
```

这些 keyframe 不作为硬边界，只作为 soft cues 放进 prompt，帮助 VLM 判断哪里可能是阶段切换，比如对齐、抓取、放置、开爪、闭爪。

- 机械臂运动停顿点
- 夹爪开合状态变化点

Step 3：把视频、原始任务、keyframes、原子动作表一起喂给 VLM

prompt 里包含：

```
1. 原始长任务描述
2. sampled multi-view video frames
3. timestamped keyframe events
4. atomic-action vocabulary
5. primitive definitions（动作具体指的什么，参考Table 8）
6. category assignment rules
7. JSON output schema
```

论文附录给出的简化 prompt 模板里，action vocabulary 包括 `grasp, place, lift, lower, transport, wrist_rotate, reorient, push, pull, press, move, rotate, wait`，还包括 object-status tags：`HOLDING` 和 `EMPTY`。

也就是说，VLM 不是自由写 caption，而是在受约束的 schema 下输出：

```
[
  {
    "subaction": "lift",
    "instruction": "lift the object upward while holding it",
    "start": 3.50,
    "end": 5.00
  }
]
```

论文明确给了这个 JSON 例子。

Step 4：语言监督的形式是什么?

LA 训练真正用的语言输入是 `instruction` 字段，而不是单独的 `subaction` label。

比如：

```
subaction: lift
instruction: lift the object upward while holding it
```

这里：

```
subaction = 结构化 primitive label，用于约束和验证
instruction = 喂给 policy 的自然语言条件
```

这个 instruction 有明确写作规则：

```
描述 action type
描述 direction
可选描述 object status
避免 object identity
避免 appearance
避免 material
避免 scene-specific details
```

论文说 instruction 要写成 compact、vision-agnostic form，描述 action type、direction 和可选 object/object-state information，同时避免外观和场景细节。

所以它不会理想地写成：

```
lift the red cup from the left side of the table
```

而更接近：

```
lift the object upward while holding it
```

或者：

```
transport the object left while keeping it upright
```

这就是 LA 里的语言监督。

Step 5：人工验证过滤

VLM 生成的是候选，不直接全收。人工会检查每个候选 segment 的：

```
temporal boundary
primitive label
natural-language instruction
```

是否互相一致。质量分低的会被过滤，例如过度切分、多个 primitive 合并、边界不准、instruction 和动作不匹配等。论文说最终保留 quality score ≥ 2 的 segment。

规模上是：

```
原始 VLA episodes: 9,560
VLM-generated LA candidates: 56,899
Final human-verified LA episodes: 33,116
```

也就是先自动生成候选，再人工过滤，最后得到 LA-33K。

最终 LA episode 大概是：

```
sample = {
    "instruction": "lift the object upward while holding it",
    "robot_states": s_t_to_s_T,
    "actions": a_t_to_a_T,
    "visual_observations": None
}
```

训练目标是：

```
instruction + proprioceptive state → action trajectory
```

原子动作设计：

Table 2 Atomic action categories organized by interaction context and motion type.

	Object manipulation	Contact interaction	Free-space motion
Gripper	grasp / place	–	–
Translation	lift / lower / transport	push / pull / press	move
Rotation	wrist_rotate / reorient	–	rotate / reorient

数据集统计：

Table 3 Dataset scale and episode granularity during LA dataset construction.

Processing Phase	Episodes	Frames	Frames / Ep.
Original VLA episodes	9,560	2,751,615	287.83
VLM-generated LA candidates	56,899	2,551,102	44.84
Final human-verified LA episodes	33,116	1,524,990	46.05

Language-Action Pretraining

Performance on MetaWorld & LIBERO & Real-world

Model	Pretrain	Success Rate (%)					Gain
MetaWorld		**Easy**	**Medium**	**Hard**	**Very Hard**	**Avg.**	
GR-1 [39]	VLA	76.6	35.3	46.0	44.0	50.5	–
SmolVLA [21]	No	87.1	51.8	70.0	64.0	68.2	–
RoboTron Mani [40]	VLA	85.5	67.7	76.7	81.0	77.7	–
Evo-1 [22]	No	89.2	76.8	77.2	79.2	80.6	–
$\pi_{0.5}$ [41]	Yes	86.1	78.2	80.0	82.0	81.6	–
Evo-depth [6]	No	83.1	84.7	**87.3**	82.4	84.4	–
ALAM [42]	VLA	89.3	83.6	85.0	82.0	85.0	–
StarVLA [16]	No	85.7	60.0	40.6	47.3	58.39	+0.00
StarVLA [16]	LA	85.1	62.2	55.0	77.3	69.91	+11.52
LA4VLA-1B	No	82.5	72.7	41.7	82.0	69.73	+0.00
LA4VLA-1B	VLA	88.6	68.2	68.3	94.0	79.78	+10.05
LA4VLA-1B	LA	**93.2**	81.8	65.0	92.0	83.00	+13.27
LA4VLA-1B	LA-VLA	91.4	90.9	66.7	98.0	86.75	+17.02
LA4VLA-1B	MixPT	88.9	**94.5**	66.7	**100.0**	**87.53**	**+17.80**
LIBERO		**Spatial**	**Object**	**Goal**	**Long**	**Avg.**	
OpenVLA [1]	VLA	84.7	88.4	79.2	53.7	76.5	–
π_0-FAST [23]	VLA	96.4	96.8	88.6	60.2	85.5	–
GR00T N1 [3]	VLA	94.4	97.6	93.0	90.6	93.9	–
π_0 [2]	VLA	96.8	98.8	95.8	85.2	94.2	–
Evo-1 [22]	No	92.7	97.7	96.3	92.3	94.8	–
Evo-depth [6]	No	95.6	99.2	95.6	91.3	95.4	–
StarVLA [16]	No	96.4	98.4	95.8	84.2	93.70	+0.00
StarVLA [16]	LA	**97.4**	99.0	96.0	87.0	94.85	+1.15
LA4VLA-1B	No	87.8	96.4	96.4	90.8	92.85	+0.00
LA4VLA-1B	VLA	93.2	**99.4**	96.8	88.2	94.40	+1.55
LA4VLA-1B	LA	93.6	98.6	97.0	92.0	95.30	+2.45
LA4VLA-1B	MixPT	93.4	98.2	98.0	**93.4**	95.75	+2.90
LA4VLA-1B	LA-VLA	95.8	97.4	**98.5**	**93.4**	**96.28**	**+3.43**

Table 5 Real-world manipulation performance under different pretraining schemes. Success rates are reported in percent. Each task is evaluated with 20 trials per policy.

Model	Pretrain	Press Button	Place Book	Place Drink	Avg.	Gain
LA4VLA-1B	No	60.0	15.0	40.0	38.3	+0.0
LA4VLA-1B	VLA	50.0	40.0	55.0	48.3	+10.0
LA4VLA-1B	LA	**85.0**	65.0	**95.0**	81.7	+43.4
LA4VLA-1B	MixPT	75.0	**85.0**	90.0	**83.3**	**+45.0**

Table 5 shows that LA pretraining strengthens real-world manipulation ability. Averaged over the three tasks, LA4VLA-1B achieves 38.3% success without pretraining, while VLA pretraining improves performance to 48.3%. LA pretraining yields a much higher average success of 81.7%, with consistent gains across all three tasks. Combining LA and VLA pretraining via MixPT further improves performance to 83.3%. These results show that the benefits of language-action pretraining extend to real-world manipulation, substantially improving success rates over both no-pretraining and VLA-pretraining baselines, with further gains when combined with VLA pretraining through MixPT.

LA-VLA: 先LA后VLA

MixPT: 混训

视觉扰动鲁棒性

Table 6 Robustness of real-world policies under Gaussian image noise. Success rates are reported in percent. Each task is evaluated with 20 trials per policy.

Model	Pretrain	Press Button	Place Drink	Avg.	Gain
LA4VLA-1B	No	40.0	15.0	27.5	+0.0
LA4VLA-1B	VLA	20.0	65.0	42.5	+15.0
LA4VLA-1B	LA	55.0	**80.0**	67.5	+40.0
LA4VLA-1B	MixPT	**65.0**	75.0	**70.0**	**+42.5**

在真实机器人上加 **Gaussian image noise**。结论很直接：**LA 和 MixPT 在视觉退化时保持更高成功率。**这支持作者的核心假设：LA 预训练减少了 policy 对视觉 shortcut 的过度依赖，使动作预测更受语言和动作 prior 控制。

指令跟随和表征分析

Condition	DAR ↑	DCS ↑	SR ↑	SS ↑
LA-pretrained	**0.99**	**0.85**	**1.46**	**0.20**
Conflicting state	0.95	0.80	1.41	0.16

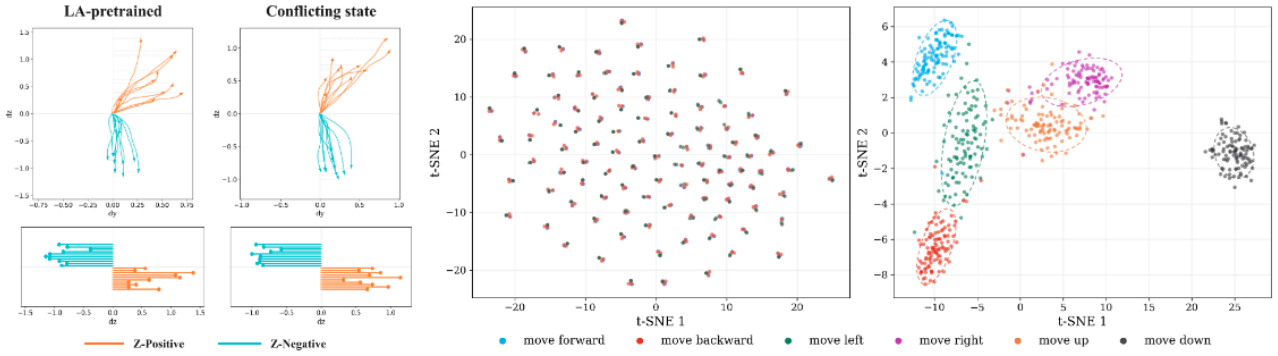


Figure 5 Direction-following behavior and representation analysis of the LA-pretrained policy. **Left panel:** qualitative direction-following under the *LA-pretrained* and *Conflicting-state* conditions for a representative pair of opposite commands along the z axis. The top row shows predicted trajectories in the dy–dz plane, and the bottom row shows endpoint projections along the tested z axis. **Right panel:** t-SNE visualization of the policy representations before action decoding, colored by direction command. The VLA-trained policy maps different commands to mixed clusters, whereas the LA-pretrained policy forms clearer command-aligned clusters.